

Lab Instructions

This lab focuses on **multithreading for I/O-bound AI pipelines**: file loading, simulated image loading, metadata extraction, error handling, throughput measurement, and producer–consumer design. Python’s threading documentation notes that threads are particularly useful for I/O-bound tasks such as file operations or network requests, while `ThreadPoolExecutor` provides a higher-level interface for running background thread tasks. Python’s `queue.Queue` is designed for safe multi-producer, multi-consumer communication between threads and implements the required locking semantics internally. [\[docs.pytorch.org\]](https://docs.pytorch.org) [\[youtube.com\]](https://www.youtube.com)

Main Goal

Students build and benchmark a threaded data loader that simulates or actually performs AI dataset loading.

By the end, students should understand:

- Why threads are useful for I/O-bound tasks.
 - How to use `ThreadPoolExecutor`.
 - How to handle errors in parallel loading.
 - How to measure throughput and speedup.
 - How to use queues for producer–consumer pipelines.
 - Why “more threads” does not always mean “faster.”
-

Lab Learning Objectives

Students will:

1. Write a **serial dataset loader** as a baseline.
2. Write a **threaded dataset loader** using `ThreadPoolExecutor`.
3. Benchmark different `max_workers` values.
4. Compute:
 - runtime

- throughput
 - speedup
 - error count
5. Handle failed/corrupted files safely.
 6. Optionally implement a **producer-consumer pipeline** using `queue.Queue`.

Part A — Full Simulated Lab Code

This version does **not require real images**, so every student can run it immediately.

See the example code

Part B — What Students Should Observe

Students should usually see:

- Serial loading is slow because each fake file waits one after another.
- Threaded loading is faster because waiting time can overlap.
- Throughput improves as workers increase.
- Speedup eventually flattens.
- Very high worker counts may stop helping or become slightly worse.

This connects directly to the idea that `ThreadPoolExecutor` is useful for I/O-bound workloads and provides a high-level interface for launching callable tasks in threads.

[\[docs.pytorch.org\]](https://docs.pytorch.org), [\[youtube.com\]](https://youtube.com)

Part C — Student Results Table

Students should fill in a table like this:

Method	Workers	Mean Time	Speedup	Throughput	Loaded	Errors
serial	1		1.00			
threaded	1					

Method	Workers	Mean Time	Speedup	Throughput	Loaded	Errors
threaded	2					
threaded	4					
threaded	8					
threaded	16					
threaded	32					

Part D — Discussion Questions

1. Did threading improve runtime compared with serial loading?
2. Which worker count was best?
3. Why did speedup flatten?
4. What resource might be the bottleneck?
 - CPU?
 - Disk (slow storage)?
 - memory?
 - scheduling overhead?
5. Why is it important to handle corrupted files?
6. What would change if the task were CPU-heavy instead of I/O-heavy?
7. How could this data loader support a final image-classification project?

Part E — Optional Real Image Folder Version

If students have an image folder, they can replace `simulated_load()` with a real image metadata loader. Install Pillow if needed.

```
pip install pillow
```

Part F — Optional Producer–Consumer Queue Version

This version is more advanced and helps students understand real system design. Python's `queue.Queue` is especially useful for safely exchanging information between multiple threads because it implements locking semantics for multi-producer, multi-consumer use cases. [\[youtube.com\]](#)

Part G — Stretch Experiment: Queue Size and Backpressure

Vary the queue size:

```
max_queue_size = 1
```

```
max_queue_size = 5
```

```
max_queue_size = 20
```

```
max_queue_size = 100
```

```
max_queue_size = 0
```

In Python, a `Queue(maxsize=0)` is effectively unbounded, while a positive `maxsize` limits how many items can be placed in the queue before producers block. [\[youtube.com\]](#)

Questions

1. What happens when the queue is very small?
 2. What happens when the queue is very large?
 3. Why might an unbounded queue be dangerous in a real AI system?
 4. How does queue size relate to memory usage?
 5. How does queue size relate to throughput?
-

Part H — Stretch Experiment: CPU Work Inside Threads

Add a CPU-heavy operation after loading:

```
def cpu_transform(x):  
    total = 0  
    for i in range(20_000):
```

```

        total += (x * i) % 97

    return total

def simulated_load_with_cpu(path):
    time.sleep(random.uniform(0.01, 0.04))

    value = random.randint(1, 1000)
    feature = cpu_transform(value)

    return {
        "path": path,
        "feature": feature,
    }

```

Then rerun the benchmark.

Expected discussion:

- Threads help less if more time is spent in CPU-heavy Python code.
- This motivates Day 3: multiprocessing for CPU-bound work.
- Students should realize that real AI pipelines often contain both I/O-bound and CPU-bound stages.

Part I — Deliverables

Please send your report to lidongvtemail@gmail.com by email. In your email, use the title “Cluster 4-July 23-your name”. Each student should submit:

1. Results Table

Must include:

- method
- number of workers

- runtime
- throughput
- speedup
- loaded count
- error count

2. One Chart

Recommended chart:

x-axis: number of workers

y-axis: throughput files/sec

3. Short Reflection

Suggested prompt:

In 1–2 paragraphs:

1. Which worker count produced the best throughput?
 2. Why did adding more workers eventually stop helping?
 3. How did you handle errors?
 4. How would this threaded loader help an AI project?
 5. What part of this lab connects to your final project?
-